



Linux Fundamental

ST5003CMD Operating Systems, Security, and Networks

*BSc (Hons) Computer Science with AI Softwarica College of IT &
E-commerce*

Student Name: Ayusha Sapkota

Student ID: 250023

Subject Leader:Rupak Rajbanshi

Table of Contents

Introduction.....	3
Stage 0: Setting Up the Text Editor.....	4
Figure 1: Writing the C source file in a text editor.....	4
Figure 2 : Verification of Generated Files Using the ls Command.....	4
Stage 1: Preprocessing.....	5
Figure 3: Preprocessing stage.....	5
Stage 2: Compilation.....	6
Figure 4: Compilation stage.....	6
Stage 3: Assembly.....	7
Figure 5: Assembling stage.....	7
Stage 4: Linking.....	8
Figure 6: generating the final executable.....	8
Program Execution.....	8
Figure 7: Program execution and output.....	8

List of Figures

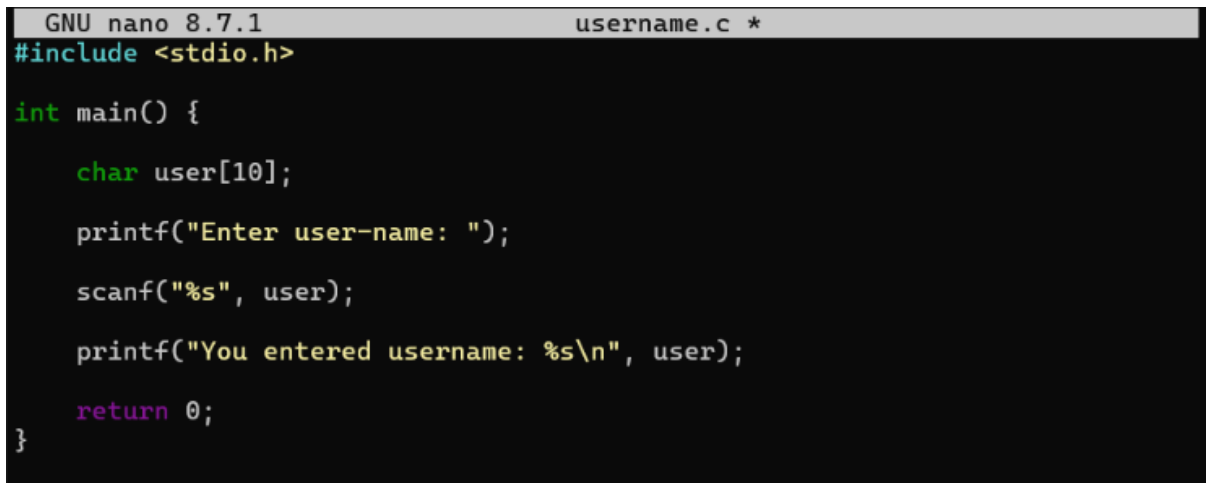
Figure 1: Writing the C source file in a text editor.....	4
Figure 2 : Verification of Generated Files Using the ls Command.....	4
Figure 3: Preprocessing stage.....	5
Figure 4: Compilation stage.....	6
Figure 5: Assembling stage.....	7
Figure 6: generating the final executable.....	8
Figure 7: Program execution and output.....	8

Introduction

As part of this exercise, we worked with a basic C program and used GCC to compile and run it. We broke the compilation process into different stages, including assembly language, to understand how the code is transformed and to check each stage individually. This makes it easier to identify and fix errors if something does not work correctly before moving to the next stage. This practically demonstrates how GCC gradually transforms C source code into a runnable executable.

Stage 0: Setting Up the Text Editor

For this exercise, the Nano text editor was used directly from the terminal to create and edit the source file. The command 'nano username.c' opens a new file named username.c in the editor, where the program shown below was written and saved:

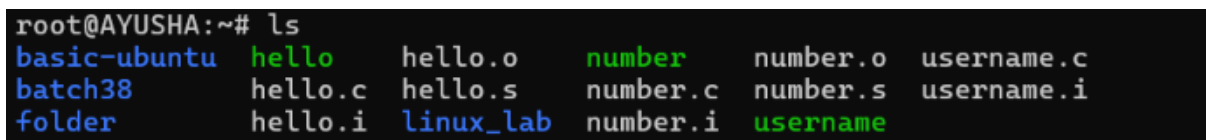


```
GNU nano 8.7.1 username.c *
#include <stdio.h>

int main() {
    char user[10];
    printf("Enter user-name: ");
    scanf("%s", user);
    printf("You entered username: %s\n", user);
    return 0;
}
```

Figure 1: Writing the C source file in a text editor

Once saved, the presence of the file was confirmed with 'ls' before moving on to the compilation stages.



```
root@AYUSHA:~# ls
basic-ubuntu  hello      hello.o    number     number.o  username.c
batch38       hello.c    hello.s    number.c   number.s  username.i
folder        hello.i    linux_lab  number.i   username
```

Figure 2 : Verification of Generated Files Using the ls Command

Stage 1: Preprocessing

The preprocessing stage prepares the C source code before compilation. It processes directives such as `#include` and expand required header files and macros. The 'username.i' file is large because the preprocessor expands the included header files. The 'head' command is used to view the first few lines and confirm that preprocessing was successful without displaying the entire file.

```
root@AYUSHA:~# gcc -E username.c -o username.i
root@AYUSHA:~# ls
basic-ubuntu  hello      hello.o    number     number.o   username.c
batch38       hello.c    hello.s    number.c   number.s    username.i
folder        hello.i    linux_lab  number.i   username
root@AYUSHA:~# head username.i
# 0 "username.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "username.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
```

Figure 3: Preprocessing stage

Stage 2: Compilation

The compiler translates the preprocessed C code into assembly language specific to the target architecture. The assembly code is stored in the .s file. We checked the generated assembly using 'head username.s'. This allows us to see how the C instructions have been represented at a lower level.

```
root@AYUSHA:~# gcc -S username.i -o username.s
root@AYUSHA:~# ls
basic-ubuntu  hello      hello.o    number     number.o   username.c
batch38       hello.c    hello.s    number.c    number.s   username.i
folder        hello.i    linux_lab  number.i    username   username.s
root@AYUSHA:~# head username.s
.file "username.c"
.text
.section      .rodata
.LC0:
.string "Enter user-name: "
.LC1:
.string "%s"
.LC2:
.string "You entered username: %s\n"
.text
```

Figure 4: Compilation stage

Stage 3: Assembly

The assembler converts the human-readable assembly code into machine code, producing an object file. At this point the code exists in binary form but has not yet been linked to any libraries. The object file's disassembly is inspected with: `objdump -d username.o`. This displayed the disassembled machine instructions contained in the object file.

```
root@AYUSHA:~# gcc -c username.s -o username.o
root@AYUSHA:~# ls
basic-ubuntu  hello      hello.o    number     number.o  username.c  username.s
batch38       hello.c    hello.s    number.c    number.s  username.i
folder        hello.i    linux_lab  number.i    username  username.o
root@AYUSHA:~# objdump -d username.o

username.o:      file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <main>:
 0:  f3 0f 1e fa                endbr64
 4:  55                        push    %rbp
 5:  48 89 e5                  mov     %rsp,%rbp
 8:  48 83 ec 20               sub     $0x20,%rsp
 c:  64 48 8b 04 25 28 00      mov     %fs:0x28,%rax
13:  00 00
15:  48 89 45 f8               mov     %rax,-0x8(%rbp)
19:  31 c0                    xor     %eax,%eax
1b:  48 8d 05 00 00 00 00      lea     0x0(%rip),%rax          # 22 <main+0x22
```

Figure 5: Assembling stage

Stage 4: Linking

The linker combines the object file with any required libraries (such as the C standard library) to produce a final, executable binary.

```
root@AYUSHA:~# gcc username.o -o username
```

Figure 6: generating the final executable

Program Execution

Finally, the compiled program is executed to confirm correct operation:

```
root@AYUSHA:~# ./username  
Enter user-name: Anna  
You entered username: Anna
```

Figure 7: Program execution and output

